Módulos e Organização

POO Essencial

Aula 03 | Programação Back-end 2

Projeto: refatoração modular da api-produtos

Ao final da aula

SENAI

Organizar

Estruturar models, utils e scripts com responsabilidades reconhecíveis.

Modularizar

Exportar e importar recursos nativos, locais e instalados pelo npm.

Modelar

Criar Produto com estado, construtor, métodos e validações.

Capacidades e evidências

- Criar módulos ES com export default e export nomeado.
- Importar módulos usando caminhos e extensões corretos.
- Instanciar objetos com new e argumentos significativos.
- Rejeitar nome vazio, preço negativo e estoque inválido.
- Encapsular regras de estoque em métodos da classe.
- Executar scripts de teste com estrutura organizada.

Roteiro da aula

Pastas

Responsabilidades

Módulos

Import e export

Classes

Dados e métodos

Prática

Refatoração da API Produtos

O código será separado em regra, utilidade e execução.

SENAI

1. Organização e módulos

Arquivos com propósito, dependências explícitas e código reutilizável

Um arquivo único cresce sem revelar responsabilidades

SENAI

Arquivo concentrado

Dados, validação, formatação e execução ficam misturados e difíceis de testar.

Módulos pequenos

Cada arquivo expõe uma responsabilidade e declara suas dependências.

Dividir arquivos não basta. A separação precisa ter significado.

Cada pasta comunica uma responsabilidade

SENAI

models

Classes que representam dados e regras centrais do domínio.

utils

Funções pequenas e reutilizáveis, sem regra de negócio específica.

scripts

Pontos de entrada para executar testes e demonstrações.

Três origens de módulos no Node.js

SENAI

Nativo

Fornecido pelo Node.js, como node:path e node:fs.

Local

Criado no próprio projeto e importado por caminho relativo.

npm

Pacote instalado e registrado nas dependências do projeto.

Mão na Massa - Mini-desafio 1: Import Básico

formatarMoeda.js e index.js

```
// formatarMoeda.js
export function formatarMoeda(valor) {
  return new Intl.NumberFormat('pt-BR', {
    style: 'currency',
    currency: 'BRL',
  }).format(valor);
}

// index.js
import { formatarMoeda } from './formatarMoeda.js';
console.log(formatarMoeda(89.9));
```

package.json e execução

```
{
  "private": true,
  "type": "module",
  "scripts": {
    "start": "node index.js"
  }
}
```

```
cd exemplos\Exemplo00-Sandbox\modulos
npm start
```

Teste sem .js no import e restaure.

Mão na Massa - Mini-desafio 2: Export Default

Produto.js

```
export default class Produto {
  constructor({ nome, preco }) {
    this.nome = nome;
    this.preco = preco;
  }
}
```

index.js

```
import Produto from './Produto.js';
import { formatarMoeda } from './formatarMoeda.js';

const produto = new Produto({
  nome: 'Mouse', preco: 89.9
});

console.log(`${produto.nome}:
${formatarMoeda(produto.preco)}`);
```

Depois, teste Produto entre chaves e restaure.

Mão na Massa - Mini-desafio 3: O Perigo da Dependência

SENAI

index.js

coordena

›

Produto.js

regra de Produto

›

formatarMoeda.js

apresentação

›

Sem retorno

fluxo em uma direção

Regra: A não pode importar B se B importa A.

SENAI

Produto.js, construtor

```
export default class Produto {
 constructor({ id, nome, preco, estoque = 0 }) {
  if (!Number.isInteger(id) || id <= 0) {
   throw new TypeError('ID deve ser um inteiro positivo.');
  }
  if (typeof nome !== 'string' || nome.trim() === '') {
   throw new TypeError('Nome é obrigatório.');
  }
  if (!Number.isFinite(preco) || preco < 0) {
   throw new TypeError('Preço inválido.');
  }
  if (!Number.isInteger(estoque) || estoque < 0) {
   throw new TypeError('Estoque inválido.');
  }
  Object.assign(this, {
   id, nome: nome.trim(), preco, estoque
  });
 }
```

retirar()

```
retirar(quantidade) {
 if (!Number.isInteger(quantidade) || quantidade <= 0) {
  throw new RangeError('Quantidade de saída inválida.');
 }
 if (quantidade > this.estoque) {
  throw new RangeError('Estoque insuficiente.');
 }
 this.estoque -= quantidade;
}
}
```

index.js

```
// index.js
const produto = new Produto({
 id: 1, nome: 'Mouse', preco: 89.9, estoque: 3
});
produto.retirar(1);
produto.retirar(5);
```

3. Evolução hands-on

Refatore a api-produtos da Aula 02, arquivo por arquivo

Passo 1: acesse a API e habilite ESM

```
cd ..\..\api-produtos
```

Passo 1: atualize package.json

```
"private": true,
"type": "module",
"scripts": {
  "start": "node app.js .env 1",
  "diagnostico": "node app.js",
  "produto-ausente": "node app.js .env 999",
  "id-invalido": "node app.js .env abc",
  "check": "node scripts/check.js"
}
```

Passo 2: crie a arquitetura modular

```
mkdir data, catalogo, models, utils, config,
scripts
```

Passo 3: crie a classe de domínio (models/Produto.js)

```
export default class Produto {
  constructor({ id, nome, preco, estoque = 0, categoria
}) {
    if (!Number.isInteger(id) || id <= 0) {
      throw new TypeError('ID deve ser um inteiro
positivo.');
    }
    if (typeof nome !== 'string' || nome.trim() === '')
{
      throw new TypeError('Nome é obrigatório.');
    }
    if (!Number.isFinite(preco) || preco < 0) {
      throw new TypeError('Preço deve ser um número
maior ou igual a zero.');
    }
    if (!Number.isInteger(estoque) || estoque < 0) {
```

Passo 3: models/Produto.js | 2 de 3

```
      throw new TypeError('Estoque deve ser um
inteiro maior ou igual a zero.');
    }
    if (typeof categoria !== 'string' ||
categoria.trim() === '') {
      throw new TypeError('Categoria é
obrigatória.');
    }
    this.id = id;
    this.nome = nome.trim();
    this.preco = preco;
    this.estoque = estoque;
    this.categoria = categoria.trim();
  }
  calcularValorEmEstoque() {
```

Passo 3: models/Produto.js | 3 de 3

```
    return this.preco * this.estoque;
  }
  calcularPrecoComDesconto(percentual) {
    if (!Number.isFinite(percentual) || percentual
< 0 || percentual > 100) {
      throw new RangeError('Desconto deve estar
entre 0 e 100.');
    }
    return this.preco * (1 - percentual / 100);
  }
}
```

Passo 4: data/catalogo.js

```
const dadosDoCatalogo = [
  { id: 1, nome: 'Teclado', preco: 120, estoque:
8, categoria: 'Periféricos' },
  { id: 2, nome: 'Monitor', preco: 950, estoque:
3, categoria: 'Vídeo' },
  { id: 3, nome: 'Mouse', preco: 80, estoque: 12,
categoria: 'Periféricos' }
];
export function listarDadosDoCatalogo() {
  return dadosDoCatalogo.map((produto) => ({
...produto }));
}
```

Passo 5: catalogo/consultas.js | 1 de 2

```
import Produto from '../models/Produto.js';
import { listarDadosDoCatalogo } from
'../data/catalogo.js';
function esperar(ms) {
  return new Promise((resolve) => setTimeout(resolve,
ms));
}
export async function buscarProdutoPorId(id) {
  await esperar(80);
  const dados = listarDadosDoCatalogo().find((produto)
=> produto.id === id);
  if (!dados) {
    throw new Error(`Produto ${id} não encontrado.`);
  }
  return new Produto(dados);
```

Passo 5: catalogo/consultas.js | 2 de 2

```
}
export async function listarCategorias() {
  await esperar(60);
  const categorias =
listarDadosDoCatalogo().map(({ categoria }) =>
categoria);
  return [...new Set(categorias)];
}
```

Passo 6: crie o utilitário (formatarMoeda.js)

```
export function formatarMoeda(valor) {
  if (!Number.isFinite(valor)) {
    throw new TypeError('Valor monetário deve ser
um número finito.');
  }
  return new Intl.NumberFormat('pt-BR', {
    style: 'currency',
    currency: 'BRL'
  }).format(valor);
}
```

Passo 7: config/ambiente.js | 1 de 3

```
const nomesObrigatorios = ['PORT', 'NOME_ALUNO',
'TURMA'];
export function carregarAmbiente(arquivoDeConfiguracao)
{
  if (arquivoDeConfiguracao) {
    try {
      process.loadEnvFile(arquivoDeConfiguracao);
    } catch {
      throw new Error(`Arquivo de configuração não
encontrado: ${arquivoDeConfiguracao}`);
    }
  }
  const ausentes = nomesObrigatorios.filter((nome) => {
    const valor = process.env[nome];
    return typeof valor !== 'string' || valor.trim() ===
'';
```

Passo 7: config/ambiente.js | 2 de 3

```
  });
  if (ausentes.length > 0) {
    throw new Error(`Configure no .env:
${ausentes.join(', ')}`);
  }
  return {
    nomeAluno: process.env.NOME_ALUNO,
    turma: process.env.TURMA,
    porta: process.env.PORT,
    ambiente: process.env.NODE_ENV ||
'development'
  };
}
export function exibirDiagnostico(configuracao)
{
```

Passo 7: config/ambiente.js | 3 de 3

```
  console.table({
    estudante: configuracao.nomeAluno,
    turma: configuracao.turma,
    projeto: 'api-produtos',
    ambiente: configuracao.ambiente,
    node: process.version,
    sistema: `${process.platform}
${process.arch}`,
    diretorio: process.cwd(),
    portaConfigurada: configuracao.porta
  });
}
```

Passo 8: scripts/check.js | 1 de 3

```
import { readdirSync } from 'node:fs';
import { join } from 'node:path';
import { spawnSync } from 'node:child_process';
function listarJavaScript(diretorio) {
  const arquivos = [];
  for (const item of readdirSync(diretorio, {
withFileTypes: true })) {
    if (item.name === 'node_modules') continue;
    const caminho = join(diretorio, item.name);
    if (item.isDirectory()) {
    arquivos.push(...listarJavaScript(caminho));
    } else if (item.isFile() &&
item.name.endsWith('.js')) {
      arquivos.push(caminho);
```

Passo 8: scripts/check.js | 2 de 3

```
    }
  }
  return arquivos;
}
const arquivos =
listarJavaScript(process.cwd());
for (const arquivo of arquivos) {
  const resultado = spawnSync(process.execPath,
['--check', arquivo], { stdio: 'inherit' });
  if (resultado.status !== 0) {
    process.exitCode = resultado.status || 1;
    break;
  }
}
```

Passo 8: scripts/check.js | 3 de 3

```
if (!process.exitCode) {
  console.log(`${arquivos.length} arquivos
JavaScript verificados.`);
}
```

Passo 9: novo app.js | 1 de 3

```
import { buscarProdutoPorId, listarCategorias } from
'./catalogo/consultas.js';
import { carregarAmbiente, exibirDiagnostico } from
'./config/ambiente.js';
import { formatarMoeda } from
'./utils/formatarMoeda.js';
async function executar() {
  try {
    const configuracao =
carregarAmbiente(process.argv[2]);
    const idSolicitado = Number(process.argv[3] || '1');
    if (!Number.isInteger(idSolicitado)) {
      throw new Error('Informe um identificador inteiro
para o produto.');
    }
    exibirDiagnostico(configuracao);
    const [produto, categorias] = await Promise.all([
```

Passo 9: novo app.js | 2 de 3

```
      buscarProdutoPorId(idSolicitado),
      listarCategorias()
    ]);
    console.log({
      produto: {
        id: produto.id,
        nome: produto.nome,
        preco: produto.preco,
        precoFormatado:
formatarMoeda(produto.preco),
        estoque: produto.estoque,
        categoria: produto.categoria,
        valorEmEstoque:
produto.calcularValorEmEstoque(),
```

Passo 9: novo app.js | 3 de 3

```
            valorEmEstoqueFormatado:
formatarMoeda(produto.calcularValorEmEstoq
ue())
        },
        categorias
      });
    } catch (erro) {
      console.error(erro.message);
      process.exitCode = 1;
    }
}
executar();
```

Passo 10: teste todas as regressões

```
npm run check
npm start
npm run produto-ausente
npm run id-invalido
npm run diagnostico
```

A arquitetura mudou, o comportamento permaneceu

```
app.js
├── config/ambiente.js
├── catalogo/consultas.js
├── data/catalogo.js
├── models/Produto.js
└── utils/formatarMoeda.js
```

Atividades 01 a 05

- Atividade01: exportar um formatador de ferramenta.
- Atividade02: proteger catálogo com cópias.
- Atividade03: construir uma classe de equipamento.
- Atividade04: proteger entrada e saída do almoxarifado.
- Atividade05: integrar catálogo, busca e formatação sem ciclo.
- Execute o estado inicial, implemente o TODO, rode o check e compare sucesso e falha no README.

Verdadeiro ou falso?

Uma classe é um conjunto de dados já preenchidos, enquanto uma instância é o molde vazio.

VERDADEIRO

FALSO

Complete o import

Se o módulo usa export default formatarMoeda, o import precisa usar chaves { }?

- SIM, as chaves são obrigatórias
- NÃO, o import default não usa chaves

Debate: onde fica a regra?

Por que desconto e validação de preço devem ficar em Produto, e não espalhados pelos scripts ou pelas rotas?

Discuta com a turma e justifique sua resposta com base em responsabilidade, coesão e acoplamento.

Clean Code da aula

- **Módulo pequeno:** uma responsabilidade reconhecível.
- **Classe justificada por estado e comportamento: Produto.**
- **Método com verbo:** atualizar, calcular, adicionar e validar.
- **Import compreensível:** dependências explícitas no topo.
- **Sem circularidade:** o grafo de módulos mantém uma direção.

Erros comuns e diagnóstico

- **Extensão ausente:** inclua .js no import local.
- **Caminho incorreto:** revise ./ e ../ a partir do arquivo atual.
- **Export incompatível:** confira default ou chaves do nomeado.
- **ESM desativado:** adicione “type”: “module”.
- **Campo privado:** acesse somente por métodos autorizados.
- **Abstração inútil:** não crie classe ou pasta sem necessidade.

Critérios de avaliação

Organização

Pastas semânticas, imports corretos e módulos pequenos.

Produto

Classe instanciada com validação e métodos funcionando.

Execução

Script de teste demonstra sucesso, falha e cálculo de estoque.

Evidências para concluir a aula

- Produto.js exportado corretamente e consumido no app.
- formatarMoeda isolado em utils.
- Três produtos criados com new.
- Preço negativo e nome vazio rejeitados.
- Estoque baixo e valor total calculados.
- Script executado pelo terminal sem imports quebrados.

Referências oficiais

Guia de módulos JavaScript

developer.mozilla.org/JavaScript/Modules

Classes JavaScript

developer.mozilla.org/JavaScript/Classes

Módulos ES no Node.js

nodejs.org/api/esm.html

Pacotes no Node.js

nodejs.org/api/packages.html

Bibliografia complementar

SENAI

Clean Code

Robert C. Martin. Princípios para código legível, responsabilidades e nomes significativos.

The Pragmatic Programmer

David Thomas e Andrew Hunt. Práticas para modularidade, manutenção e redução de acoplamento.